



Önálló laboratórium beszámoló

Távközlési és Mesterséges Intelligencia Tanszék

Név: **Pálos Ferenc**

palosferenc@proton.me

Neptun-kód: **U1OTPN**

Szakirány: **Mérnök informatikus szak**

Konzulens: **Ladóczki Bence**

ladoczki.bence@vik.bme.hu

Konzulens:

Téma címe: Website Fingerprinting vizsgálata Tor hálózaton

Feladat:

A Tor egy népszerű anonimizáló hálózat, amely napi több millió felhasználó adatvédelmét szolgálja több ezer, önkéntesek által üzemeltetett proxy segítségével. A hálózat célja, hogy elrejtse a kommunikáció forrását és célját, így védve a felhasználókat a megfigyeléstől, nyomkövetéstől és cenzúrától. Ugyanakkor az egyik jelentős fenyegetés a Website Fingerprinting (WF) támadás, amely során a támadó a belépési pont forgalmának megfigyelésével következtethet a meglátogatott weboldalakra. A feladat célja annak vizsgálata, hogy a WF támadások mennyire jelentenek valós fenyegetést a Tor felhasználói számára, és hogy eltérő forgalmi környezetekben (baseline, obfs4), valamint ismeretlen forgalom mellett hogyan változik a modellek teljesítménye.

Ennek részeként elvárás a valós forgalmi minták gyűjtése szimulált környezetben, etikus keretek között, az adatok előkészítése a gépi tanulási feldolgozáshoz, valamint egységes feature tér és top-10 kiválasztási stratégiák kialakítása, hogy a modellek összehasonlíthatóak legyenek különböző forgatókönyvekben.

A feladat második részeként két eltérő modellcsalád kidolgozása és kiértékelése szükséges: egy klasszikus gépi tanulási megoldás (Random Forest) és egy mélytanulási beágyazás-alapú modell (Triplet MLP + KNN). A kiértékelés egységes metrikákkal (pontosság, macro-F1), több véletlen maggal és ábragenerálással történjen, valamint a domain shift és a zero-shot beállítások hatását is számszerűsíteni kell.

Tanév: 2025/26. tanév, II. félév

1. A laboratóriumi munka környezetének ismertetése, a munka előzményei és kiindulási állapota

1.1. Bevezető

Ez a beszámoló a Tor hálózaton végzett website fingerprinting (WF) feladatot tárgyalja. A Tor célja a felhasználók anonimitásának megőrzése, ugyanakkor a belépési ponton látható forgalmi mintázatok alapján egy támadó a meglátogatott weboldalakra következtethet. A WF gyakorlatban is releváns, mert a webes tartalmak és a hálózati környezet időben változik, továbbá a Tor ökoszisztémában megjelennek forgalom-elfedést célzó technikák is (pl. obfs4). Ez a projekt ezért nemcsak zárt világú, hanem zero-shot és nyílt világú forgatókönyvekben is vizsgálja a modellek teljesítményét.

A WF valós környezetben is releváns, és Toron mért forgalom alapján értékelhető támadói teljesítményt mutat[1]. A vizsgálat fókusza a domain shift (baseline vs. obfs4), a zero-shot általánosítás és a nyílt világú ismeretlen forgalom kezelése.

A munka célja egy reprodukálható, végponttól végpontig futtatható feldolgozási lánc felépítése volt: a PCAP állományokból kiindulva a jellemzők kinyerésén és a modellek tanításán át egészen a kiértékelési metrikákig és ábrákig. A vizsgálat két eltérő modelleszaládot hasonlít össze: egy klasszikus gépi tanulási megoldást (Random Forest) és egy mélytanulási beágyazás-alapú modellt (Triplet MLP + KNN). A szakasz célja, hogy megadja a tágabb kontextust és rögzítse, miért ez a megközelítés indokolt a témában.

1.2. Elméleti összefoglaló

Website fingerprinting során a támadó nem a tartalmat, hanem a metaadatokat figyeli (csomagméret, irány, érkezési idők), és ezekből próbálja a látogatott weboldalt azonosítani. A feladat tipikusan felügyelt osztályozás: minden weboldal egy osztály, a minták pedig a weboldal-látogatások forgalmi lenyomatai. Zárt világú beállításban csak ismert osztályok léteznek, nyílt világban viszont megjelenik egy gyűjtő „Other” kategória, amely az ismeretlen webhelyeket fedi le. A zero-shot eset a gyakorlatban fontos: a modell olyan forgalomra kerül kiértékelésre (pl. obfs4), amelynek eloszlása eltér a tanítástól.

A domain shift jelensége miatt a baseline-on tanult modellek obfs4 forgalmon romolhatnak, ezért a kiértékelési keretben külön szerepet kap a baseline/obfs4 összevetés és a zero-shot beállítás.

A WF megoldások két fő irányba sorolhatók. Az egyik a kézzel tervezett, aggregált jellemzők használata, ahol egy kapcsolatot jellemző statisztikák (csomagszámok, bájtstatisztikák, időköz- és méretstatisztikák, időtartam, irányváltások) kerülnek a modell bemenetére. A másik a mélytanulási megközelítés, amely a jellemzőteret maga tanulja, például beágyazásokon alapuló osztályozással[2]. A jelen munka mindkét irányt vizsgálja: a Random Forest a statisztikus jellemzők robusztus baseline-ja, míg a Triplet MLP beágyazást tanul, amelyre KNN osztályozás épül.

A kiértékeléshez a pontosság és a macro-F1 a fő metrikák, mert több osztály esetén a teljesítmény kiegyensúlyozott megítélését adják. Nyílt világban külön fontos az „Other” osztály recall értéke is, hiszen ez mutatja meg, hogy az ismeretlen forgalom mennyire válik el a zárt világú osztályoktól. Ezek a fogalmak adják a további fejezetek értékelési keretét.

1.3. A munka állapota, készültségi foka a félév elején

A félév elején nem állt rendelkezésre átadott kód, adatgyűjtés vagy korábbi mérési eredmény, és a témában korábban nem végeztem önálló kutatást. A kiindulási állapot a feladatkiírás és a munkaterv adta, valamint az egyetemi kurzusokból szerzett alapismereteim (gépi tanulás, hálózati forgalom menedzsment). Ennek megfelelően a munka nulláról indult, a mérési infrastruktúra, a feldolgozási pipeline és a kiértékelési keretrendszer felépítésével.

2. Az elvégzett munka és az eredmények ismertetése

2.1. Áttekintés

A beszámoló a Tor hálózaton végzett weboldal ujjlenyomat-készítés (Website Fingerprinting, WF) feladattal mutatja be, két eltérő modelles család összehasonlításával: klasszikus gépi tanulás (Random Forest) és mélytanulás (Triplet MLP + KNN). A vizsgálat a zárt világú (baseline, obfs4), a zero-shot és a nyílt világú (open-world) forgatókönyvekben méri a teljesítményt. A végső értékelés a futtatott Python pipeline által számolt metrikákon és a generált ábrákon alapul. A Random Forest erős baseline és obfs4 teljesítményt ad, míg a Triplet MLP a zero-shot beállításokban kedvezőbb általánosítást mutat. A dokumentum részletesen bemutatja az adatgyűjtési és reprodukálhatósági lépéseket is, hogy a pipeline végponttól végpontig újrafuttatható legyen.

2.2. Célkitűzések és hozzájárulások

A projekt célja egy reprodukálható WF pipeline kialakítása és két modelles család objektív összehasonlítása a Tor forgalomban jelentkező domain shift és ismeretlen forgalom kezelésére. Ennek keretében:

- a PCAP állományoktól a kiértékelési metrikákig egy végig automatizált feldolgozási láncot raktam össze,
- aggregált folyamjellemzőket definiáltam és egységes 21-dimenziós feature teret alkalmaztam,
- *shared* és *optimized* top-10 feature track-eket vezettem be,
- egységes ábragenerálást biztosítottam a kiértékelési JSON-okból,
- az open-world „Other” osztály kezelését és a zero-shot beállításokat külön mérési számként kezeltem.

2.3. Adatok és forgatókönyvek

A mérés három adatdoménen történik: baseline Tor forgalom, obfs4 Tor forgalom, valamint az open-world „Other” kategória. A fő forgatókönyvek:

- **Baseline (closed-world):** standard Tor böngészés.
- **Obfs4 (closed-world):** obfs4 pluggable transport által elfedett forgalom.
- **Zero-shot:** baseline-on tanított modellek tesztelése obfs4 forgalmon.
- **Open-world:** „Other” kategória hozzáadása az ismeretlen oldalak modellezésére.

A zárt világú osztályokat rétegzett 80%/20% tanító/teszt felosztással értékelem. Az open-world „Other” osztályt ezzel szemben 50%–50% arányban bontom tanító/teszt részre, mert itt a cél nem az adott „Other” webhelyek pontos felismerése, hanem az ismeretlen forgalomra való általánosítás. A nagyobb „Other” teszt rész valóságosabb „ismeretlen” mintaállományt ad, csökkenti a recall becslés zaját, és akkor is értelmezhető méretű tesztet hagy, ha az „Other” mintaszám kisebb.

2.4. Adatgyűjtés és előfeldolgozás

Az adatgyűjtés automatizált headless böngészéssel és tcpdump-pal történik, aktív Tor SOCKS proxy és ControlPort mellett. A SOCKS proxy a Tor hálózatra irányítja a böngésző forgalmát, míg a ControlPort a Tor példány vezérlését teszi lehetővé (pl. új áramkör kérése, állapotlekérdezés). A zárt világú baseline/obfs4 forgalmat a `scripts/collection/collector.py` (futtató: `run_collector.sh`) gyűjti, míg az open-world „Other” adatokat a `server_collect_other.py` szkript (futtató: `run_server_collect_ot`) állítja elő és a `tor_dataset/other_tor/` könyvtárba menti. A gyűjtés környezetét opcionálisan környezeti változók (pl. `TOR_WF_PCAP_DIR`, `TOR_WF_INTERFACE`, `TOR_WF_GUARD_IP`, `TOR_WF_CAPTURE_DURATION`, `TOR_WF_WARMUP_DURATION`) szabályozzák. Itt a `TOR_WF_INTERFACE` adja meg a tcpdump által figyelt hálózati interfészt (pl. `eth0` vagy `wlan0`), a `TOR_WF_WARMUP_DURATION` pedig a tényleges mérés előtti bemelegítési időt másodpercben, amely a Tor/circuit stabilizálását szolgálja.

1. táblázat. Aggregált folyamjellemzők csoportjai és példák.

Csoport	Jellemzők (példák)
Csomagszámok	total_pkts, in_pkts, out_pkts
Bájtstatisztikák	total_bytes, in_bytes, out_bytes, in/out ratio
Időköz-statisztikák	iat_mean, iat_std, iat_median, iat_p90
Méretstatisztikák	size_mean, size_std, size_median, size_p90
Időtartam és dinamika	duration, direction_changes

2.5. Feldolgozási pipeline és jellemzők

A pipeline három lépésből áll. **(1)** A PCAP fájlokból kiolvasom az egyes csomagokat, és csomagonkénti idősorokat készítek (mikor jött a csomag, mekkora volt, milyen irányban haladt). **(2)** Ezekből az idősorokból egy-egy kapcsolatot összefoglaló, aggregált jellemzőket számolok. Ilyenek például az összes be- és kimenő csomagszám, a be- és kimenő bájtok összege, a csomagok közti érkezési idők statisztikái (átlag, szórás, kvantilisek), a kapcsolat időtartama, illetve az irányváltások száma. **(3)** A jellemzőkön tanítom és értékelem a modelleket. Az alap 21 jellemzőből minden futásban top-10 készletet választok, így a modellek összehasonlítása egységes dimenziószámon történik. A jellemzők főbb csoportjait a 1. táblázat foglalja össze.

2.6. Modellek és kiértékelés

Random Forest. 300 döntési fából álló modell, stabilizált top-10 jellemzőkiválasztással és három véletlen maggal (42, 52, 62). A három mag segítségével a tanító/teszt felosztást és a modell tanítását háromszor ismétlem, majd átlagot és szórást számolok.

Triplet MLP + KNN. A Triplet MLP 32 dimenziós beágyazást tanul. A Triplet Margin Loss arra kényszeríti a modellt, hogy az azonos weboldalhoz tartozó minták beágyazása közel legyen, míg a különböző weboldalaké távol. A beágyazásokon $k = 5$ KNN osztályozást végzek.

Feature track-ek. *shared* track-ben egy közös top-10 készletet választok ki a mutual information alapján. *optimized* track-ben forgatókönyvenként külön top-10 készlet készül (RF: több futásból stabilizált fontosági rangsor, DL: mutual information).

Mélytanulási beállítások. A Triplet MLP 30 epochon tanul standardizált bemeneten, BatchNorm és Dropout rétegekkel, hogy csökkentse a túltanulást.

2.7. Kiértékelési metrikák

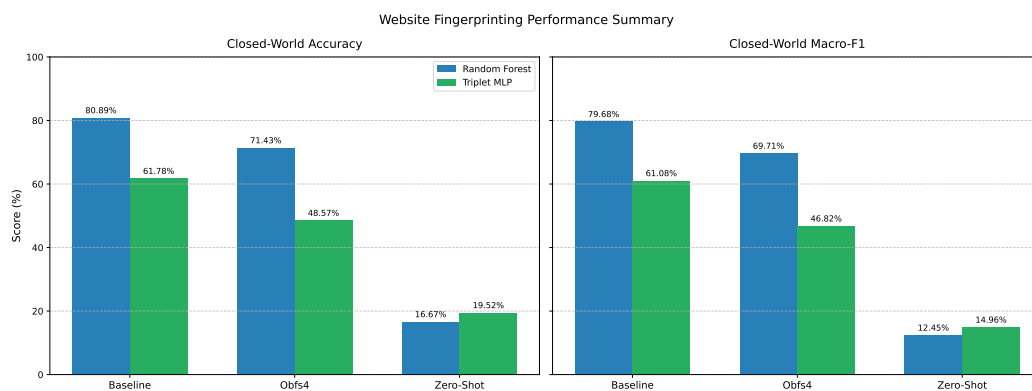
A teljesítményt pontossággal és macro-F1 értékkel mérem. A pontosság a helyes predikciók aránya, a macro-F1 pedig az osztályonként számolt F1-értékek átlaga:

$$\text{Acc} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}_i = y_i], \quad \text{MacroF1} = \frac{1}{C} \sum_{c=1}^C \text{F1}_c. \quad (1)$$

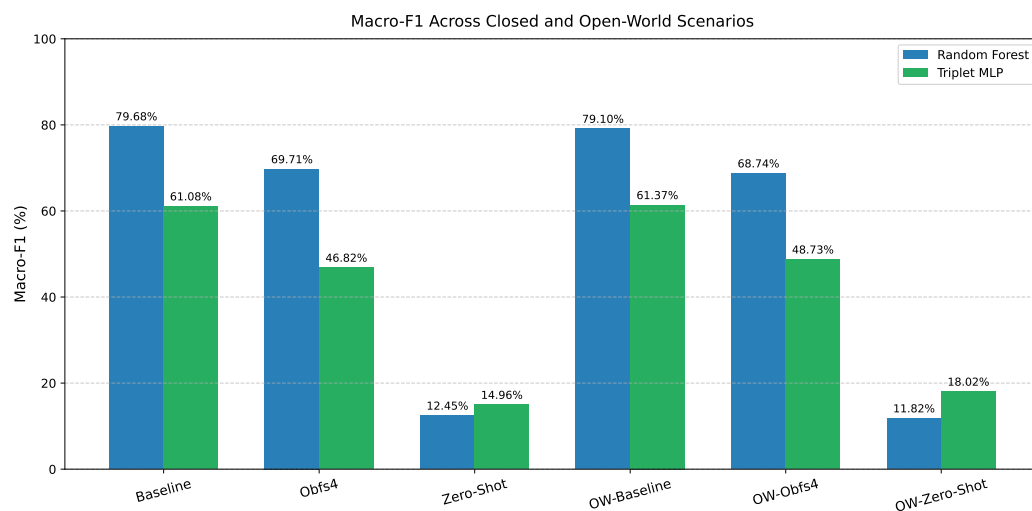
Open-world esetben külön vizsgálom az „Other” osztály recall értékét is, mivel ez mutatja, hogy az ismeretlen forgalmat mennyire képes a modell leválasztani.

2.8. Ábrák és vizuális összegzés

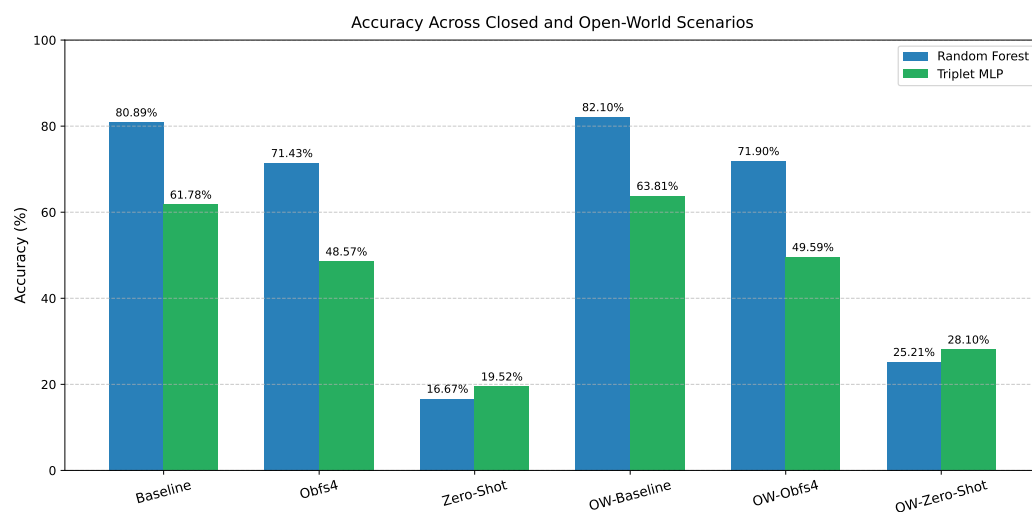
A `generate_all_figures.py` szkript a mentett metrikákból generálja a végső ábrákat, ezért az ábrák és a táblázatok ugyanabból a kiértékelési kimenetből készülnek. Az ábrák célja, hogy a fő trendeket gyorsan összegezze, majd részletesebb bontást adjon forgatókönyv, feature track és osztályszintű viselkedés szerint.



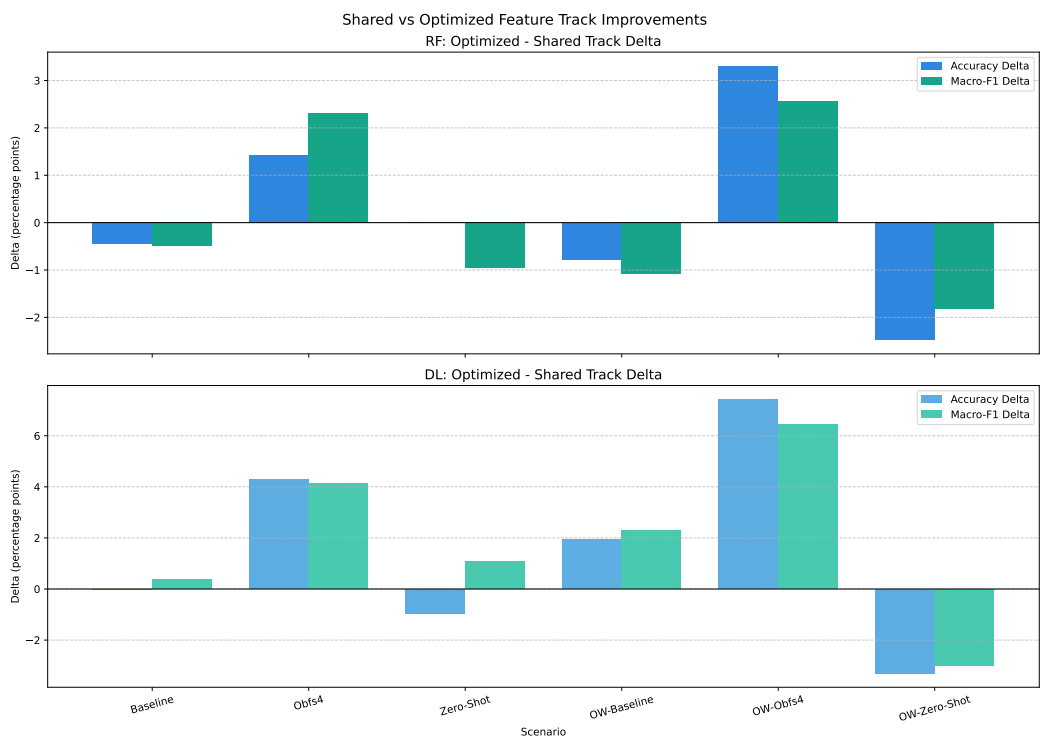
1. ábra. Zárt világú teljesítmény-összegzés (pontosság és macro-F1) a két modellre.



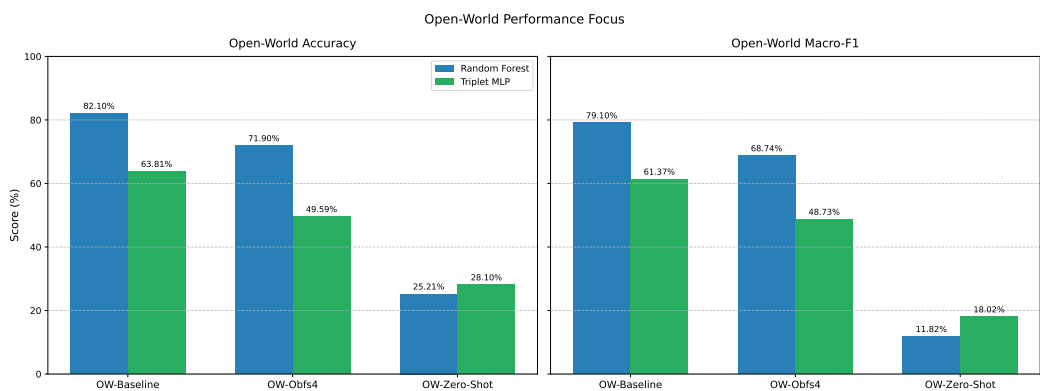
2. ábra. Macro-F1 alakulása minden forgatókönyvben, zárt és nyílt világú esetekben.



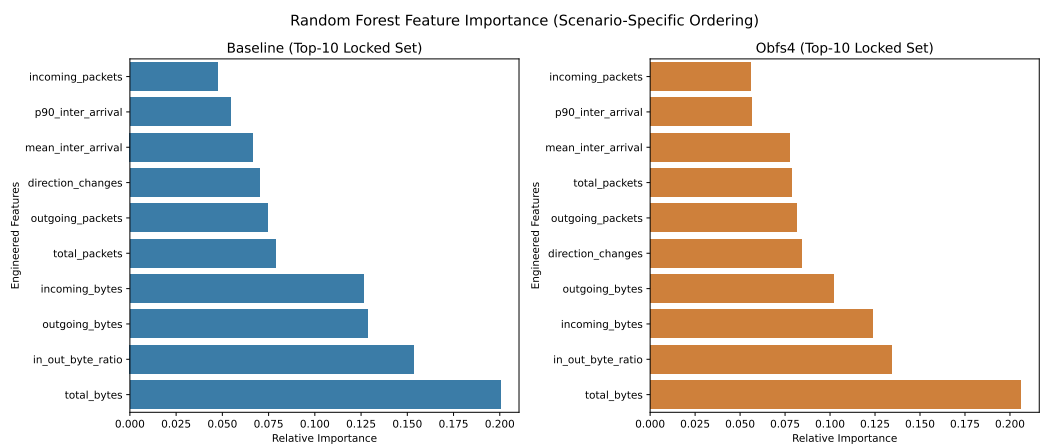
3. ábra. Pontosság alakulása minden forgatókönyvben, zárt és nyílt világú esetekben.



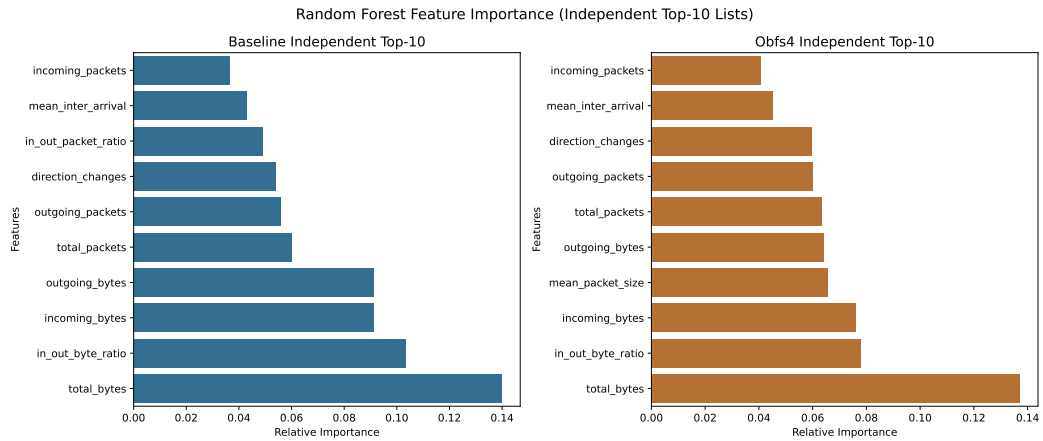
4. ábra. A *shared* és *optimized* track-ek közti különbségek (delta) pontosságban és macro-F1-ben.



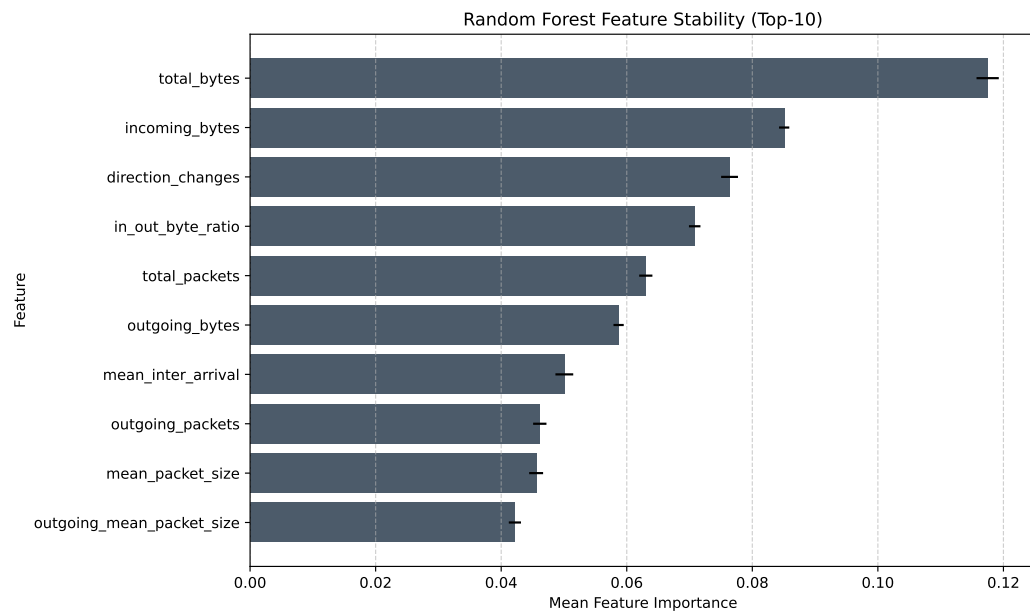
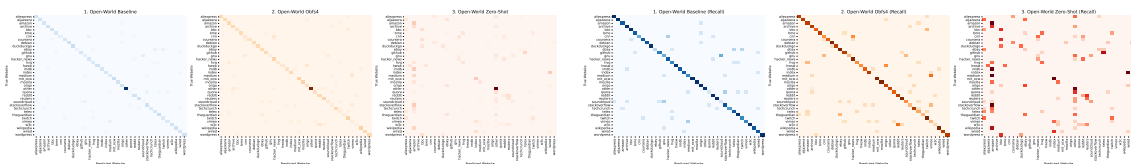
5. ábra. Nyílt világú forgatókönyvek összehasonlítása.



6. ábra. A top-10 jellemzők fontossága a közös (shared) kiválasztás mellett.



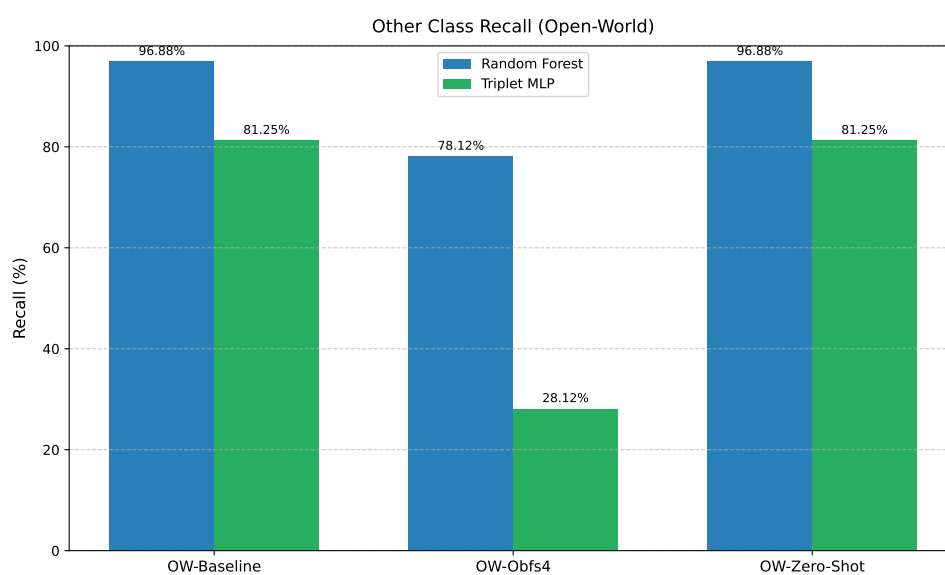
7. ábra. Forgatókönyvenként optimalizált top-10 jellemzők összehasonlítása.

8. ábra. A top-10 jellemzők stabilitása Random Forest esetén (átlag $\pm$ szórás a több futásból).

(a) Alap konfúziós mátrixok.

(b) Recall-normalizált változatok.

9. ábra. Konfúziós mátrixok a főbb forgatókönyvekben (shared track, seed=42).



10. ábra. Az „Other” osztály recall értékei az open-world forgatókönyvekben.

2.9. Kísérleti beállítások

A tanítás és kiértékelés rétegzett tanító/teszt felosztással történik. A zárt világú osztályoknál 80%/20% arányt használok, míg az open-world „Other” osztályt 50%–50% arányban bontom az ismeretlen forgalomra való általánosítás hangsúlyozására. A stabilitást három véletlen mag biztosítja (42, 52, 62). Az eredményeket átlag $\pm$ szórás formában közlöm. A metrikák a pontosság (accuracy) és a macro-F1. A zero-shot beállításban a baseline adatokon tanított modelleket obfs4 teszt adatokon értékelem. A táblázatok a *shared* track eredményeit mutatják.

2.10. Reprodukálhatóság és futtatás

A pipeline fő lépései az alábbi parancsokkal futtathatók:

- `python src/extract_all_features.py`
- `python src/evaluate_all_rf.py`
- `python src/evaluate_all_dl.py`
- `python src/generate_all_figures.py`

2.11. Eredmények

2.12. Elemzés és megvitatás

A Random Forest zárt világú baseline és obfs4 környezetben egyértelműen magasabb pontosságot és macro-F1-t ér el, ami a kézzel tervezett aggregált jellemzők hatékony kihasználását jelzi (lásd 1 és 3). A zero-shot beállításban mindkét modell teljesítménye drasztikusan romlik, különösen open-world esetben, ami a domain shift egyik legfontosabb gyakorlati kihívását mutatja (lásd 2 és 5). A Triplet MLP ugyanakkor stabilabb zero-shot értékeket ad (Standard Zero-Shot: DL 23.01% vs. RF 15.08%), ami a beágyazások általánosíthatóságára utal. Az *optimized* track-ek több forgatókönyvben javítást hoznak, de a javulás nem egyenletes, ezért a *shared* beállítás jó kompromisszumot ad a konszisztencia és teljesítmény között (lásd 4). A stabil top-10 lista azt sugallja, hogy a leginformatívabb aggregált jellemzők robusztusak a tanító/teszt felosztás változására (lásd 8). Az open-world forgatókönyvekben a recall-normalizált konfúziós mátrixok jól kiemelik, hogy az „Other” osztályt a modellek gyakran sikeresen külön választják a zárt világú osztályoktól, ugyanakkor ez nem feltétlenül jelent magas precíziót (lásd 9b és 10).

2.13. Korlátok

- A vizsgálat aggregált folyamjellemzőkre épül, ami részben elfedheti a finom időbeli mintázatokat.
- Az open-world „Other” osztály sokféle weboldal forgalmát fogja össze, ezért a modell nehezebben választja el a zárt világú osztályoktól, ami csökkentheti a precíziós mutatókat.
- A top-10 jellemzők statikus kiválasztása miatt a ritkább, de informatív mintázatok elveszhetnek.

2.14. Következtetések és jövőbeli munka

A projekt igazolja, hogy a klasszikus és mélytanulási modellek eltérő erősségekkel rendelkeznek a Tor WF feladatban. A Random Forest magas baseline teljesítményt ad, míg a Triplet MLP zero-shot helyzetekben

2. táblázat. Zárt világú (Standard) forgatókönyvek eredményei.

Forgatókönyv	RF Acc.	RF Macro-F1	DL Acc.	DL Macro-F1
Baseline	80.44 $\pm$ 0.96	79.76 $\pm$ 0.99	61.19 $\pm$ 2.22	60.23 $\pm$ 2.47
Obfs4	73.33 $\pm$ 1.78	71.88 $\pm$ 1.57	50.64 $\pm$ 2.59	49.48 $\pm$ 2.70
Zero-Shot	15.08 $\pm$ 1.37	10.83 $\pm$ 1.57	23.01 $\pm$ 2.59	18.54 $\pm$ 2.64

3. táblázat. Nyílt világú (Open-World) forgatókönyvek eredményei.

Forgatókönyv	RF Acc.	RF Macro-F1	DL Acc.	DL Macro-F1
OW-Baseline	80.93±0.84	79.11±0.68	62.00±1.32	60.37±0.76
OW-Obfs4	74.10±1.59	71.03±1.65	48.90±2.58	48.46±1.78
OW-Zero-Shot	22.73±1.79	10.41±1.00	27.27±1.79	18.42±0.41

jobb általánosítást mutat. A jövőben a drift-robosztus tanítás (az időben változó forgalmi eloszláshoz igazított tréning, pl. időablakos validálás vagy domain-invariáns reprezentációk) és az adaptív újratanulás (drift detektálás után periodikus vagy online frissítés friss adatokból) lehet indokolt. Emellett a finomabb idősoros jellemzők integrálása is ígéretes (csomag-idősor, burst-mintázatok, n-gram vagy embedding alapú leírás), mert ezek érzékenyebben ragadják meg a forgalom dinamikáját. Végül érdemes a modellfrissítés költség-hatékonyágát is vizsgálni: mennyi új adat és számítási idő szükséges adott teljesítmény-nyereséghez, és milyen frissítési gyakoriság adja a legjobb kompromisszumot.

2.15. Összefoglalás

A félév során egy végponttól végpontig futtatható WF pipeline-t építettem, amely a PCAP állományoktól a metrikákig és ábrákig egységes feldolgozást biztosít. A kísérletekben a Random Forest erős baseline és obfs4 teljesítményt adott, míg a Triplet MLP a zero-shot beállításokban stabilabb általánosítást mutatott. Az open-world vizsgálatok rávilágítottak arra, hogy az „Other” osztály leválasztása lehetséges, de a precízió korlátos marad. A legfontosabb tanulság, hogy a domain shift kezelése kulcsfontosságú, ezért a következő lépésekben drift-robosztus tréning és finomabb idősoros jellemzők bevezetése indokolt.

3. Irodalom, és csatlakozó dokumentumok jegyzéke

3.1. A tanulmányozott irodalom jegyzéke

- [1] Giovanni Cherubin, Rob Jansen, and Carmela Troncoso. *Online Website Fingerprinting: Evaluating Website Fingerprinting Attacks on Tor in the Real World*. In 31st USENIX Security Symposium (USENIX Security 22), pp. 753–770, 2022. https://www.usenix.org/system/files/sec22summer_cherubin.pdf
- [2] Payap Sirinam, Mohsen Imani, Marc Juarez, and Matthew Wright. *Deep Fingerprinting: Undermining Website Fingerprinting Defenses with Deep Learning*. In Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS '18), pp. 1928–1943, 2018. DOI: <https://doi.org/10.1145/3243734.3243768>

3.2. A csatlakozó dokumentumok jegyzéke

A csatlakozó anyagok a projekt könyvtárában találhatók (forráskód, gyűjtőszkriptek, ábrák és mérési kimenetek).